

HP : 0 / 100

03:37

FPS : 46.8

Seed : 3910578937

Enemy : 650

爽快感のあるゲーム体験を求めて

日本工学院専門学校蒲田校

ゲームクリエイター科四年制

浅川 咲人 「アサカワ サクト」

ポートフォリオ



もくじ

- ・ 作品概要 開発環境 3P ~ 4P
- ・ 作品紹介 5P ~ 14P

作品概要

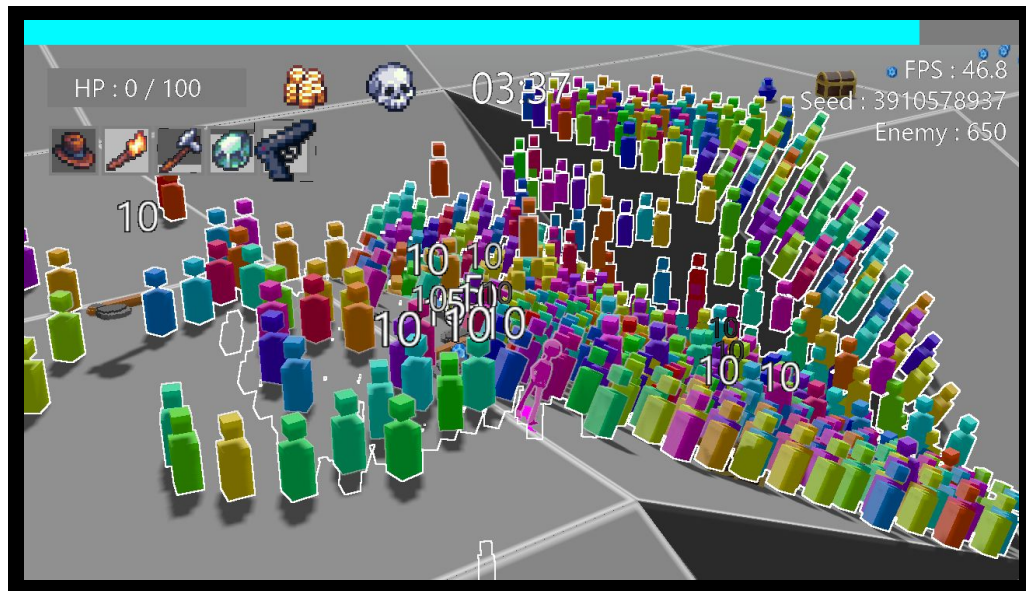
タイトル 「GIGA Survivor」

ジャンル：ローグライク

開発人数：1人

開発期間：2026/6～現在

開発環境：自作エンジン



[紹介動画](#)



開発環境

使用言語	C++ / HLSL
GraphicsAPI	DirectX12
ライブラリ	Assimp / DirectXTex / ImGui / nlohmann/json
開発ツール	VisualStudio2026 / GitHub / Blender / FireAlpaca

HP : 4 / 100



04:41

FPS : 60.0

Seed : 1398600261

Enemy : 89

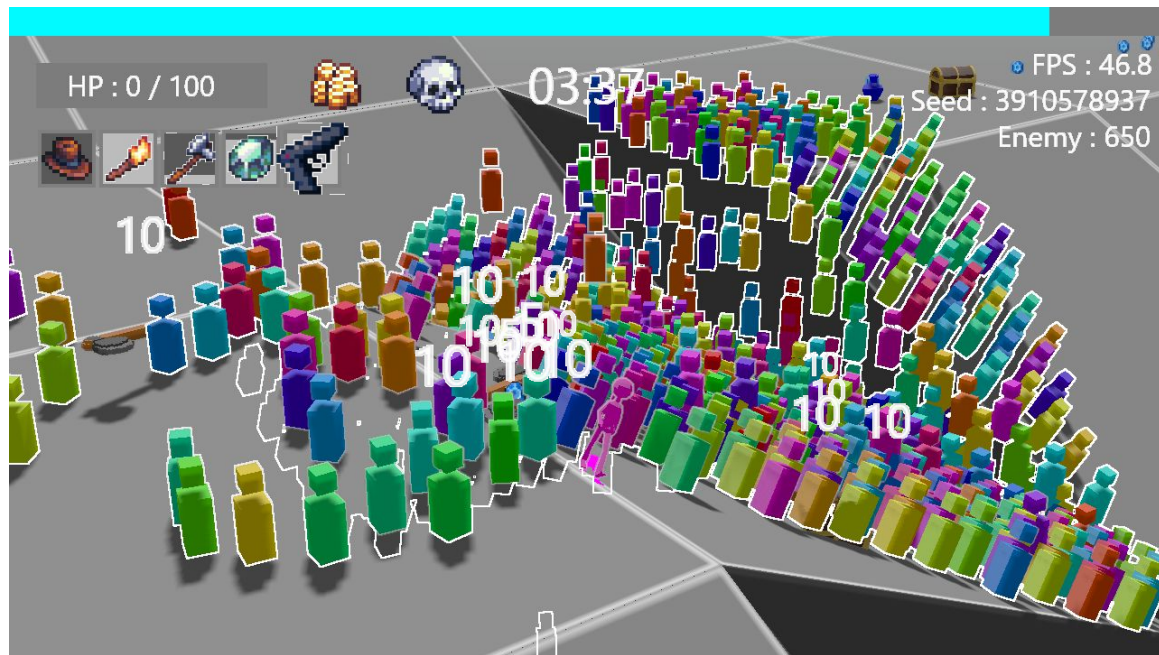


作品介绍



目標

繰り返し遊びたくなるような「爽快感のあるゲーム体験」を作成する目標があり
それを果たすためには、**画面を埋め尽くす大量の敵**が必要だと考えました。



問題発生

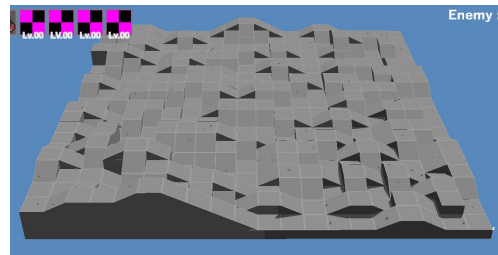
20 x 20のマップブロックを用意して敵を生成すると
Debugビルドの敵100体生成で、FPSが 60→20 まで落ちてしまった。

原因はColliderManagerに総当たり方式を採用していたことで
敵1体につき、400個のマップブロックと当たり判定を取っており

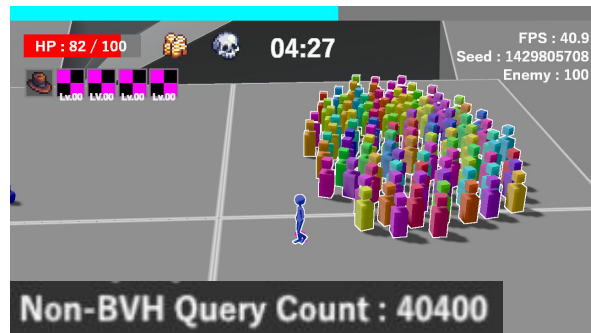
$400 \times 100 = 40000$ と **判定の組み合わせ数が敵数に比例して増加**

今後、敵や投射物などの動的なオブジェクトを増やす予定だったため
総当たり方式では限界があると判断し、軽量化に踏み切った。

x20 z20 のマップ ↓



敵100体のスクショとGuiに出した計算量↓



分割法の比較

先生から空間分割法を使うよう助言を受け、二年次に見た資料から様々な分割法を探し最終的に「K-d Tree」と「BVH (Bounding Volume Hierarchy)」を候補として挙げた。

調べて比較した結果 ↓

「K-d Tree」

メリット

- ・ 静的なオブジェクト同士の判定が得意
- ・ オブジェクトがあまり動かない場合は高速

デメリット

- ・ 動的オブジェクトに弱く、敵が毎フレーム移動する場合、再構築や再挿入のコストが問題になりやすい

「BVH」

メリット

- ・ 動的Colliderにも対応しやすく、移動したColliderだけ更新する設計にできる
- ・ ColliderManagerのAABBをそのまま木に入れられるので、衝突判定と相性が良い

デメリット

- ・ k-d Treeより実装が少し複雑 毎フレーム全再構築すると重くなる可能性がある

今回は 敵(動的) と マップ(静的) の判定を軽くしたいため「BVH」を採用。

Static BVHの採用

BVHを採用することにしたが、Dynamic BVHは動的オブジェクトに合わせた木の更新処理が必要になるため、実装範囲が広くなると考えた。

だが、Static BVHであれば、マップの配置後に木を構築し、その後は更新せずに探索へ利用できると考え、まずStatic BVHを試作して効果を確認しその後、本実装をすることにした。

方式	実装・更新コスト	本作へ適用判断
総当たり	なし (0)	100体で20FPSまで低下で限界が来たため不採用
K-d Tree	木構造が単純で簡単 (低)	静的なオブジェクトに強く動的オブジェクトに適さないと判断したため不採用
Dynamic BVH	毎フレーム木の更新が必要 (高)	実装コストが高く、登録するマップが動かないため動的更新機能を実装する必要がなかったため不採用
Static BVH (採用)	初期化時の一回のみ (低)	動かないマップの形状がAABBなので最適であり実装コストも高くないため採用

判定候補の絞り込み

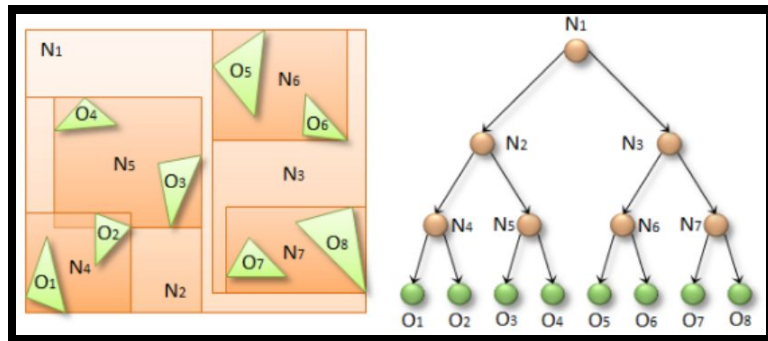
階層的境界ボリューム

マップブロック群を内包する境界ボリュームで木構造を構築し探索時に敵Colliderと交差しないノードを早期離脱させる。

1. ルートノードから交差判定を開始
2. 交差しないノードの子要素はスキップ
3. 衝突の可能性あるマップブロックのみを抽出

このプロセスで計算量を減少させる。

BVHの概念図↓



登録から判定までの流れ

衝突判定パイプライン↓

1. マップ登録

マップ生成時に静的な
マップブロックAABB情報を取得

2. 木の構築

ゲーム開始時にStatic BVHを
事前構築(再更新なし)

3. BVH探索

毎フレーム敵Colliderで木を探索
して判定候補を抽出

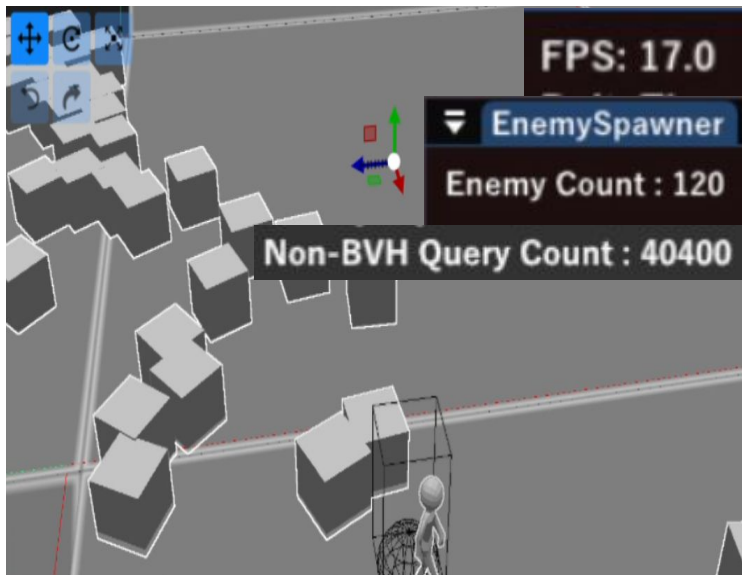
4. 詳細判定

ColliderManagerが絞り込ま
れた候補と衝突判定を取る

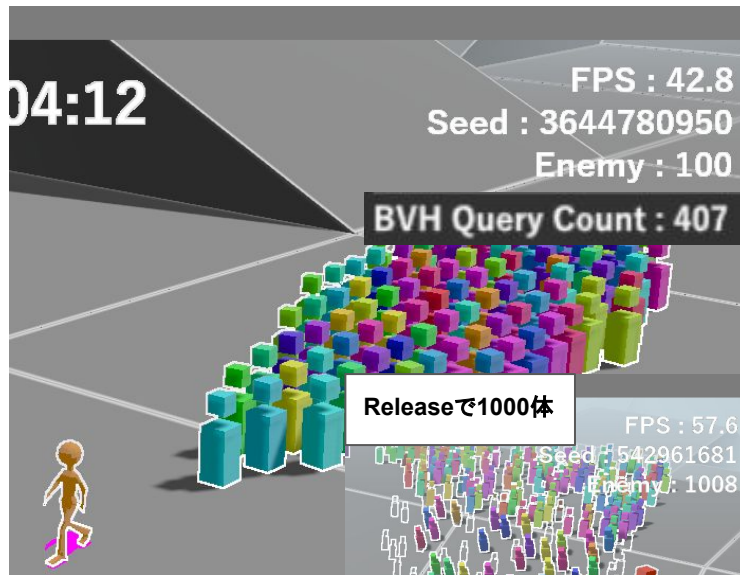
検証結果

Static BVHの探索適用により、DebugビルドのImGui上で判定候補数が1フレームあたり、40000回 → 400回 近くまで**激減**していることが確認できた。
また、FPSも17 → 42 まで向上しており軽量化に成功した。

総当たり↓ (2026/06/30)時点



Static BVH↓ (2026/08/16)時点

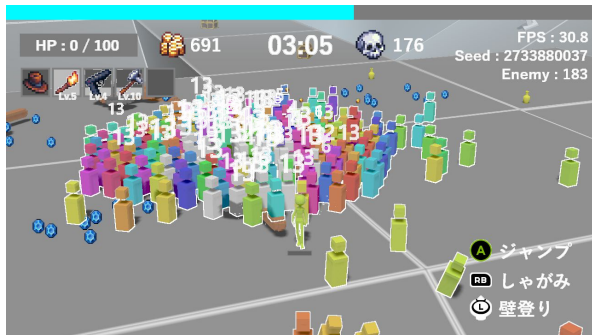


恩恵

敵を大量に出せるようになったので、纏めて攻撃した時のエフェクトやダメージが大量に出る気持ち良さやアイテムが大量に落ちた時の達成感がかなり増しました。

結果として「爽快感のあるゲーム体験」にかなり近づいたのではと感じました。

大量の敵を一掃する動画



まとめ

最適化の実装で得た知見

データ特性に基づく問題の単純化:

静的データ (地形) と動的データ (敵) を分離して捉えることで
過剰に複雑なシステムを作らずに、実効性のある解決策 (Static BVH) を選択できた。

ボトルネックに対する局所的アプローチ:

最初から全体を汎用フレームワーク化するのではなく
問題となっている負荷の高い処理に絞って軽量化を実施した。

今後行うこと

軽量化に成功したとはいえ、Debugビルドの敵100体でFPS40は目標の60FPSを満たしていない。
敵同士の排斥処理が次のボトルネックである可能性があるため、まずCPU時間を計測して
問題が確認できた場合は、Dynamic BVHなどの方式検討する。